Behavioural & Soft Skills

Italics = Reflection
Bold = Main Point

---

1. Tell me about yourself. (Give a concise personal introduction).

Hi — I'm Steven, a full-stack developer with three years' consulting experience and a master's in engineering.

I've delivered end-to-end web and mobile solutions — from API design and databases to frontend UX and deployment.

Recently I took a focused break to upskill in cloud, generative AI and modern frameworks, and I've been building hands-on projects to apply those skills.

I'm looking to join a team where I can own features end-to-end and help deliver scalable, user-focused products, including pragmatic AI integrations.

---

2. Why do you want to be a software/full-stack developer?.

I became a full-stack developer because software combines everything I enjoy about engineering: problem solving, systems thinking and fast feedback loops.

My background exposed me to multiple engineering disciplines, but software lets me prototype solutions quickly and measure real impact.

In my last role I led a UX demo that cut customer queries by 31% and built an ETL pipeline that removed manual syncing.

I enjoy working across teams to turn customer needs into robust code and I want to continue doing that here.

---

3. What are your greatest strengths and weaknesses?

<u>Strengths</u>

Commitment to continuous development — I actively seek feedback and look for ways to improve my skills. I attend conferences and meetups, such as React and Django events, and take targeted courses to keep my knowledge current and practical.

Team integration — I bring positive energy to teams and make a deliberate effort to build strong working relationships. I enjoy mentoring colleagues and organising informal touchpoints, such as virtual lunch sessions, to help maintain team cohesion, even in remote environments.

Weaknesses

Over-analysing problems too early — I sometimes spend too much time researching before validating my assumptions with others.

To improve this, I now sanity-check my understanding with a colleague early on and time-box initial research to around 30–60 minutes. That balance of quick validation followed by focused investigation has noticeably reduced wasted effort.

Judging the right level of documentation — While documentation is important for handovers and complex systems, over-documenting can slow delivery.

To address this, I follow a lightweight rule: document intent and key interfaces (such as environment variables, core data entities, and high-level sequences) and favour concise diagrams or short READMEs over lengthy documents. I also include an owner and review date so documentation remains useful and up to date.

---

4. Describe your biggest professional achievement to date.

Project references: Oman Foreign Ministry

My biggest achievement was leading the stage-one delivery of a mobile app and admin portal.

I translated stakeholder needs into a prioritised backlog, designed a resilient data model and an automated ETL to sync WordPress content, and mentored two juniors through paired programming and code reviews.

We delivered the MVP, the client commissioned follow-on work, and the juniors became productive contributors.

It showed I could own both technical design and team delivery end-to-end.

---

5. What is the hardest problem you've solved, and how did you solve it?.

Project references: obs-etl-k8s-demo-project

On an observability/ETL demo an Airflow container refused to start and a downstream service broke.

Logs hinted at a Redis/version mismatch, but changing the Redis env var didn't fix it. I systematically ruled out causes: inspected logs, tried config changes, used GitHub and Stack Overflow, and used

LLM tools for ideas.

Eventually I audited image tags and discovered upstream image availability/policy issues and migrated to the community image. The deployment then succeeded.

The key was methodical elimination, targeted research and being willing to change the image source.

---

6. Tell me about a time you failed or received criticism and what you learned.

Project references: Barnfield Caravans

Early in my career I implemented multiple form features at once to be efficient, but that caused an overrun on the first ticket. Because the client had Kanban access, their confidence dropped and they paused the project.

I was told I should have consulted stakeholders before deviating from the ticket approach. I learned that transparency and alignment are as important as efficiency.

Now I raise proposed deviations early and update the backlog with the team before changing course.

---

7. How do you handle tight deadlines or stressful situations?

Project references: Bare Dating

I focus on three things: prioritisation, breaking work into manageable tasks, and clear communication.

I identify the highest-value deliverables, decompose them into small tasks, and keep the team and stakeholders updated so blockers are visible. For example, when a client publicly announced features before they were ready.

We prioritised the demo-critical items, ran quick check-ins, and kept the client informed — which let us deliver the essentials without surprises.

---

8. Give an example of working effectively under pressure.

Project references: Alternative Commercial Finance

I had a “poison-chalice” project where I was the primary developer and had to reverse-engineer the domain from code because migrations wouldn’t run.

I learned Node.js and GraphQL quickly, instrumented the code to trace data flows, produced an ERD, and organised customer sessions to validate assumptions.

Despite runtime and SQL compatibility issues, I delivered the features and the domain report so the team could continue.

The lesson: rapid learning + structured investigation + proactive stakeholder communication lets you succeed when you're under pressure.

---

9. Describe a conflict you had with a teammate and how you resolved it.

I inherited a module refactored into a large pure-SQL query, with no tests or notes. The author argued it was faster; I argued it reduced readability and maintainability.

After discussion I pointed out that I had to maintain the code, so we agreed I'd revert the main branch for immediate maintainability.

Then I wrapped the SQL in a documented, tested function and added a performance test so we kept the speed benefit but regained clarity.

We resolved it with benchmarks, documentation and a pragmatic compromise.

---

10. How do you receive and act on feedback?

Project references: ai_tool-risk_mitigation-demo-project, obs-etl-k8s-demo-project

I always thank people for feedback, then turn it into an action plan I can validate.

After interviews I received two pieces of feedback: I needed more exposure to production debugging and more concrete, impact-focused examples.

I built two projects: an Observability & ETL demo to practise deployment and monitoring, and a secure AI integration demo plus an interview Q&A repo to surface concrete examples.

I set success criteria, sought peer review and iterated — feedback became a measurable improvement loop.

---

11. How do you prioritise tasks when given multiple priorities?

I use a value/effort baseline then apply pragmatic checks: hard deadlines, blockers, resource availability and dependencies.

High-value, low-effort items come first, but deadlines bump priorities up. I check for blockers (credentials, a third-party, or a subject-matter expert) and group related tasks where it reduces work.

Finally, I communicate the proposed prioritisation to stakeholders and re-evaluate regularly as new information arrives.

---

12. Give an example of when you took initiative or leadership in a project.

Project references: Oman Foreign Ministry

On one project the "nearest embassy" logic was undefined. I documented options, but that slowed decisions, so I organised a face-to-face meeting with the client and project team.

Walking everyone through the trade-offs led to a quick clarification and a simple practical implementation. Development resumed with no further delay.

It showed that bringing the right people together can be the fastest way to resolve ambiguous requirements.

---

13. How do you communicate complex technical concepts to non-technical stakeholders?

I focus on outcomes, use simple analogies and visuals, and avoid jargon.

For example, explaining an ETL sync to non-technical stakeholders I called the website the "source of truth" and compared updates to a news feed refresh, then showed a simple diagram of the flow. That kept the discussion on user impact and got fast buy-in.

---

14. Have you ever disagreed with a manager's decision? What did you do?

My manager proposed splitting boilerplate into multiple repos and automating admin-page generation.

I raised operational concerns — extra CI, secrets and limited practical benefit — and presented examples where modularisation helps. I respected the decision, helped implement it, and suggested a pilot approach.

The initiative was later dropped, but the process taught me to present trade-offs clearly and propose measurable pilots before wide adoption.

---

15. How do you ensure you stay current with new technologies? (Learning habits)

I combine courses, community and hands-on practice.

Recently I've completed courses in Django, Node.js, microservices, Python for data science, NoSQL, ETL, containers, observability, application security and generative AI.

I attend meetups and conferences, browse GitHub and Reddit, and set aside time for focused projects — for example an ETL pipeline or an AI demo — so I apply learning immediately.

I also share findings with peers through pair programming and talks.

---

16. Where do you see yourself in 5 years?.

In five years I see myself as a senior or lead full-stack engineer owning end-to-end features and helping shape technical direction.

I want to be shipping scalable systems — particularly AI integrations — while mentoring others and improving engineering practices across the team.

---

17. What motivates you at work?

I'm motivated by solving meaningful problems that deliver business value, learning new technologies, and collaborating with others.

Seeing a feature go from idea to production and actually help users is especially rewarding and often leads to further improvements.

---

18. How do you maintain work-life balance?

I set clear boundaries, prioritise work so I focus on the highest-value tasks during the day, and make time for exercise and recovery.

I also communicate capacity with the team so we plan realistically — and when peaks happen I schedule recovery time afterwards.

---

19. Describe a time you had to learn something quickly to complete a task.

Project references: 2PD

I needed to add GraphQL to an admin portal fast.

I took a short course on GraphQL fundamentals, then built a quick prototype using Apollo Server with Express to learn schemas, resolvers and request flow.

I applied that pattern to the codebase, mapped data structures to GraphQL types, added resolvers and tests, and documented the model.

That let me deliver the functionality within the deadline and hand over maintainable code.

---

20. How would your colleagues describe you?

References: Eleni, Jordan, Audrius

Colleagues would describe me as dependable, detail-focused and collaborative — someone who raises the bar on quality while being easy to work with.

They'd say I'm logical, calm under pressure, and willing to mentor others. As one colleague put it, "Steven is a great presence to have around."